

LABORATORY PRACTICAL REPORT

Lab 2 — File Upload and Command Execution Security Assessment

Special Topics in Information Security

Targets: DVWA (Damn Vulnerable Web Application) and OWASP Mutillidae II

Field	Detail
Student Name	Sadeeq
Student ID	FCP/CCS/22/1004
Institution	Federal University Dutse, Jigawa State, Nigeria
Programme	Cyber Security Science (Final Year)
Assessment	Lab 2 of 5 — Merged Practical Lab Series
Course Access Window	24 August 2026 – 6 September 2026 (WAT)
Assessment Deadline	Saturday, 29 August 2026, 11:59 PM WAT
Environment	Isolated VM — DVWA (127.0.0.1:8081) and OWASP Mutillidae II (127.0.0.1:8082)
Report Date	28 August 2026

1. Executive Summary

This report documents a controlled security assessment of two intentionally vulnerable training applications — DVWA and OWASP Mutillidae II — focused on two areas: file upload validation and OS command execution (command injection). A development team's claim that these features are secure was evaluated using harmless marker files and minimal-impact, non-destructive command markers, strictly within the authorised isolated VM environment.

The assessment found that, at default (low) security settings, both applications accepted uploaded files whose content did not match their declared type or extension, and both executed operating-system commands injected through diagnostic input fields. Neither weakness required credentials beyond normal application login, and neither was exploited beyond the minimum needed to prove the vulnerability existed. A comparison against stronger security settings showed that command-injection filtering improved earlier (at DVWA's Medium level) than upload validation, which only began enforcing real content-type checks at DVWA's High level. Root-cause remediations are provided for both vulnerability classes.

1.1 Summary of Findings

#	Finding	Application(s)	Severity (Observed Context)
F1	Upload validation trusts client-supplied extension/MIME type; no content inspection performed	DVWA, Mutillidae	High (root cause of unrestricted file type acceptance)
F2	Uploaded files retained original filename with no server-side renaming	DVWA, Mutillidae	Medium
F3	Upload directory stored inside public webroot	DVWA	High (in combination with F1, enables stored-file retrieval)
F4	Client can specify server-side upload destination directory	Mutillidae	Medium-High (design weakness)
F5	User input concatenated directly into an OS shell command	DVWA, Mutillidae	Critical (remote command execution as www-data)
F6	Command filtering is blacklist-based rather than allowlist/parameterised	DVWA	Medium (mitigated at Medium/High level, not eliminated architecturally)

2. Scope and Rules of Engagement

All testing was performed exclusively against locally installed, isolated instances of DVWA and OWASP Mutillidae II running inside a training virtual machine. Neither application was exposed publicly at any point.

2.1 In-Scope Targets

- File upload: DVWA File Upload module; OWASP Mutillidae II upload challenge.
- Command execution: DVWA Command Injection module; OWASP Mutillidae II DNS Lookup module.

2.2 Test Material

- Learner-created harmless marker files (one text file, one PNG image).
- Non-destructive command markers: echo string output and the whoami identity command only.

2.3 Constraints Observed

- No executable file was uploaded at any point.
- No malware, reverse shell, or external network callback was used.
- No persistence mechanism (scheduled task, account, listener, web shell) was created.
- No privilege escalation was attempted.
- No destructive command or access to unrelated files/credentials/systems occurred.
- All activity remained local to the isolated VM (127.0.0.1:8081 for DVWA, 127.0.0.1:8082 for Mutillidae).

3. Methodology

Testing followed the ten required hands-on phases specified in the lab brief: evidence preparation, upload baseline, validation comparison, storage assessment, command baseline, minimal execution proof, impact assessment, secure-mode comparison and retest, remediation, and cleanup. Burp Suite Community Edition was used as an intercepting proxy to capture raw HTTP requests and responses; sha256sum and the file utility were used to fingerprint marker files.

4. Phase 1 — Evidence Preparation

Two harmless marker files were created in the VM and fingerprinted prior to any upload activity, establishing an unaltered baseline for later comparison.

Evidence #	File	Type	Size	SHA-256
E1	marker.txt	ASCII text	60 bytes	c0b4a7c6a716682755ee396c0b6e31535b3b64603d1db6fc711f7c924c59ff08
E2	marker.png	PNG image, 100x100, 8-bit RGB	292 bytes	bbaf3b444bb28de3aa83120f43123da756dd0b995acf9df8d8d9d32f05169d2

```

kali@kali: ~/lab2_evidence/markers
Session Actions Edit View Help
(kali@kali)-[~/lab2_evidence/markers]
$ cp /usr/share/pixmaps/some-icon.png marker.png 2>/dev/null || echo "use any small p
ng you have"
use any small png you have

(kali@kali)-[~/lab2_evidence/markers]
$ cp ~/Desktop/Screenshot\ 2026-08-26\ 155002.png 2>/dev/null || echo "use any small
png you have"
use any small png you have

(kali@kali)-[~/lab2_evidence/markers]
$ for f in marker.txt marker.png; do
for> echo "= $f ="
file "$f"
stat -c "Size: %s bytes" "$f"
sha256sum "$f"
echo
for> done
= marker.txt =
marker.txt: ASCII text
Size: 60 bytes
c0b4a7c6a716682755ee396c0b6e31535b3b64603d1db6fc711f7c924c59ff08 marker.txt

= marker.png =
marker.png: cannot open 'marker.png' (No such file or directory)
stat: cannot statx 'marker.png': No such file or directory
sha256sum: marker.png: No such file or directory

```

Evidence E1 — marker.txt created and fingerprinted (SHA-256, size, MIME type).

```

kali@kali: ~/lab2_evidence/markers
Session Actions Edit View Help
$ cd ~/lab2_evidence/markers
python3 -c "
from PIL import Image
img = Image.new('RGB', (100, 100), color=(73, 109, 137))
img.save('marker.png')
"

(kali@kali)-[~/lab2_evidence/markers]
$ for f in marker.txt marker.png; do
echo "= $f ="
file "$f"
stat -c "Size: %s bytes" "$f"
sha256sum "$f"
echo
done
= marker.txt =
marker.txt: ASCII text
Size: 60 bytes
c0b4a7c6a716682755ee396c0b6e31535b3b64603d1db6fc711f7c924c59ff08 marker.txt

= marker.png =
marker.png: PNG image data, 100 x 100, 8-bit/color RGB, non-interlaced
Size: 292 bytes
bbaf3b444bb28de3aa83120f43123da756dd0b995acf9df8d8d9d32f05169d92 marker.png

(kali@kali)-[~/lab2_evidence/markers]

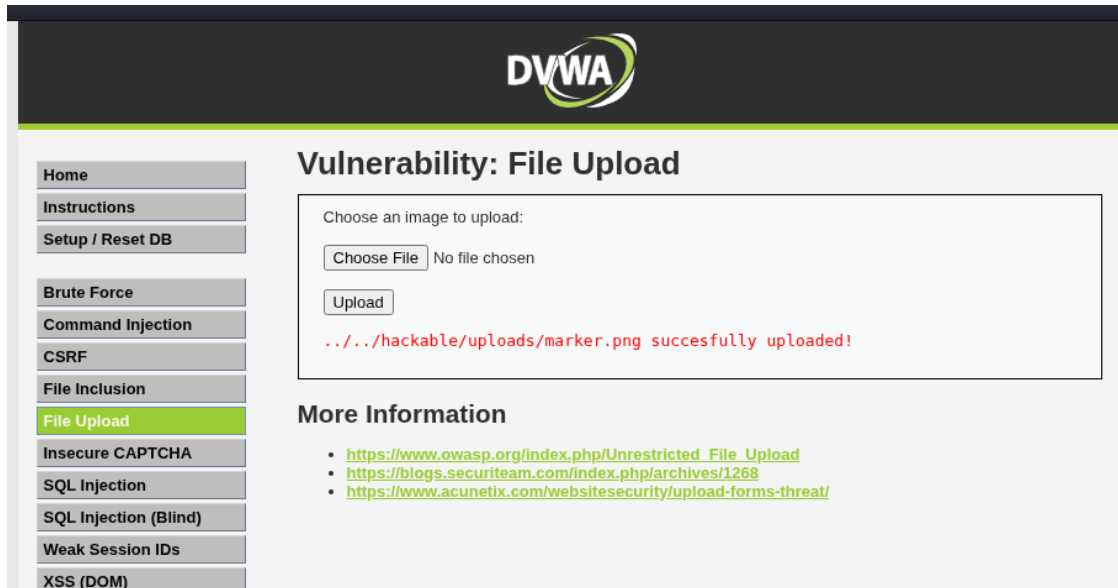
```

Evidence E2 — marker.png generated via Python/Pillow and fingerprinted.

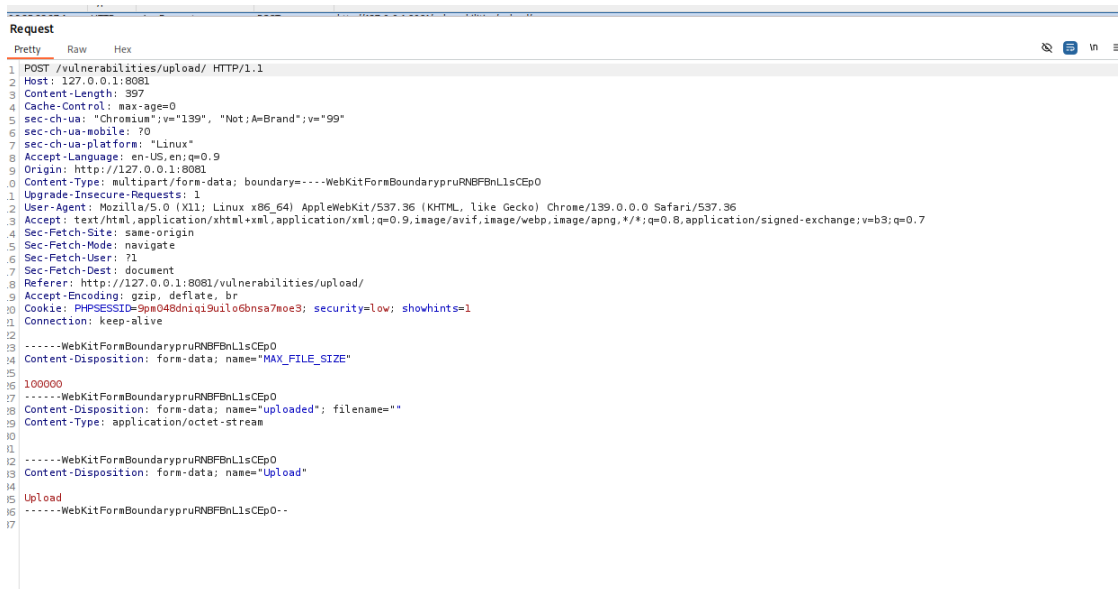
5. Phase 2 — Upload Baseline

marker.png was uploaded unmodified to both applications at default (low/hosed) security to record normal, expected behaviour.

5.1 DVWA Baseline Upload



Evidence E3 — DVWA reports successful upload to .../hackable/uploads/marker.png.



Evidence E4 — Raw multipart POST request captured in Burp Suite for the DVWA baseline upload.

Request

```

1 POST /vulnerabilities/upload/ HTTP/1.1
2 Host: 127.0.0.1:8081
3 Content-Length: 684
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="139", "Not;A=Brand";v="99"
6 sec-ch-ua-mobile: 0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1:8081
10 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryTjTvoMfV5aIDPBi
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1:8081/vulnerabilities/upload/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=9pn048dn1q19u1o6bnsa7moe3; security=low
21 Connection: keep-alive
22 -----WebKitFormBoundaryTjTvoMfV5aIDPBi
23 Content-Disposition: form-data; name="MAX_FILE_SIZE"
24 100000
25 -----WebKitFormBoundaryTjTvoMfV5aIDPBi
26 Content-Disposition: form-data; name="uploaded"; filename="marker.png"
27 Content-Type: image/png
28
29 PNG
30
31 IHDPdyEIDATxiDAA A@AYÀ x**ÀE:Mç'Iau;à0I
32
33
34
35
36
37
38
39
40
41

```

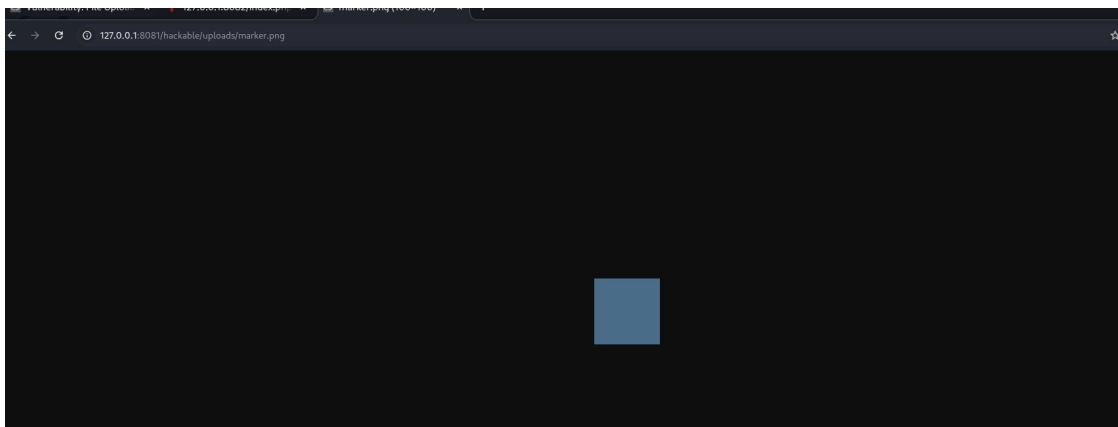
Response

```

1 HTTP/1.1 200 OK
2 Date: Thu, 27 Aug 2026 10:18:24 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Tue, 29 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 4260
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13
14 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
15 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
16 <html xmlns="http://www.w3.org/1999/xhtml">
17
18 <head>
19   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
20
21   <title>
22     Vulnerability: File Upload :: Damn Vulnerable Web Application (DVWA) v1.10
23   </title>
24
25   <link rel="stylesheet" type="text/css" href="../../../dvwa/css/main.css" />
26
27   <link rel="icon" type="image/ico" href="../../../favicon.ico" />
28
29   <script type="text/javascript" src="../../../dvwa/js/dvwaPage.js">
30
31   </script>
32
33 </head>
34
35 <body class="home">
36   <div id="container">
37
38     <div id="header">
39
40       
41
42     </div>

```

Evidence E5 — Full request/response pair (200 OK) for the DVWA baseline upload, showing security=low cookie.



Evidence E6 — Direct browser retrieval of the uploaded file confirms it renders correctly and is publicly reachable at /hackable/uploads/marker.png.

5.2 Mutillidae Baseline Upload

[Home](#)
[Login/Register](#)
[Toggle Hints](#)
[Show Popup Hints](#)
[Toggle Security](#)
[Enforce SSL](#)
[Reset DB](#)
[View Log](#)
[View Captured Data](#)

Upload a File

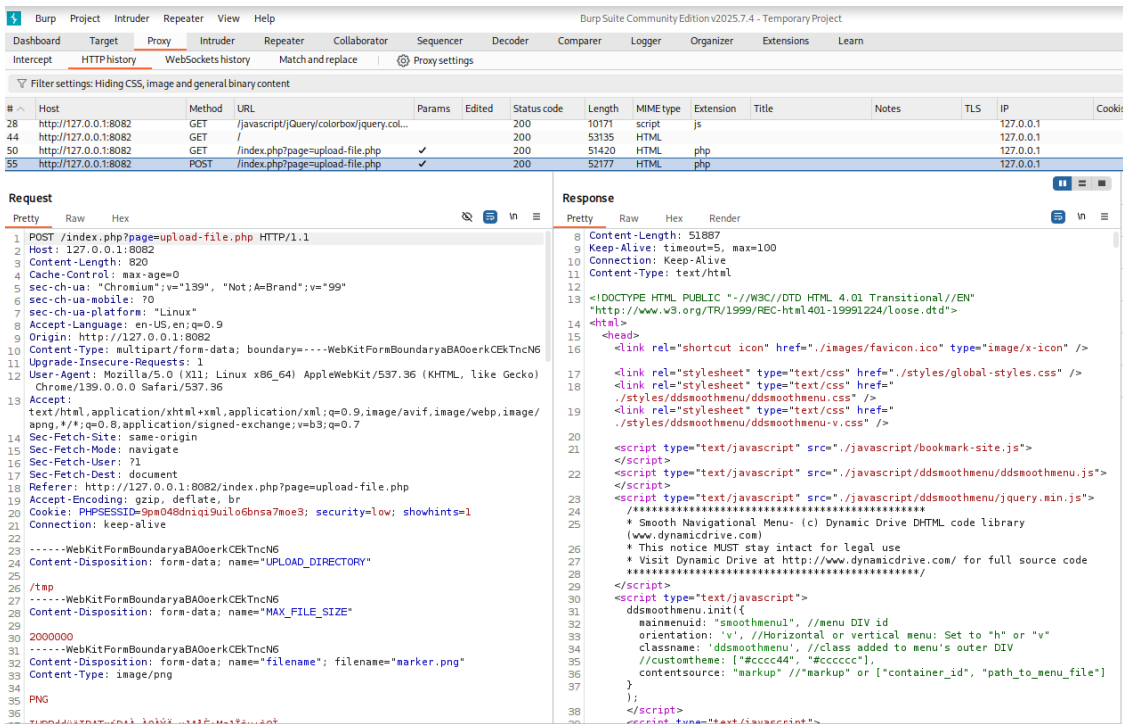
File uploaded to /tmp/phpAS32N7
 File moved to /tmp/marker.png
 Validation not performed

Original File Name marker.png
 Temporary File Name /tmp/phpAS32N7
 Permanent File Name /tmp/marker.png
 File Type image/png
 File Size 292 Bytes

Please choose file to upload

Filename 

Evidence E7 — Mutillidae confirms upload with full metadata and explicitly states "Validation not performed."



The image shows a Burp Suite capture of a POST request to /index.php?page=upload-file.php. The request is a multipart/form-data submission. The response is an HTML page with a status of 200. The response body contains a message indicating the file was uploaded and moved, and that validation was not performed. The response also includes a redirect to a page with a file named 'marker.png'.

Request:

```

1 POST /index.php?page=upload-file.php HTTP/1.1
2 Host: 127.0.0.1:8082
3 Content-Length: 820
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="139", "Not;A=Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1:8082
10 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryBA0erkCEKtncN6
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1:8082/index.php?page=upload-file.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=9pm048dn1q19u1o6bnsa7moe3; security=low; showhints=1
21 Connection: keep-alive
22
23 -----WebKitFormBoundaryBA0erkCEKtncN6
24 Content-Disposition: form-data; name="UPLOAD_DIRECTORY"
25 /tmp
26
27 -----WebKitFormBoundaryBA0erkCEKtncN6
28 Content-Disposition: form-data; name="MAX_FILE_SIZE"
29 2000000
30 -----WebKitFormBoundaryBA0erkCEKtncN6
31 Content-Disposition: form-data; name="filename"; filename="marker.png"
32 Content-Type: image/png
33
34 PNG
35
36 -----WebKitFormBoundaryBA0erkCEKtncN6

```

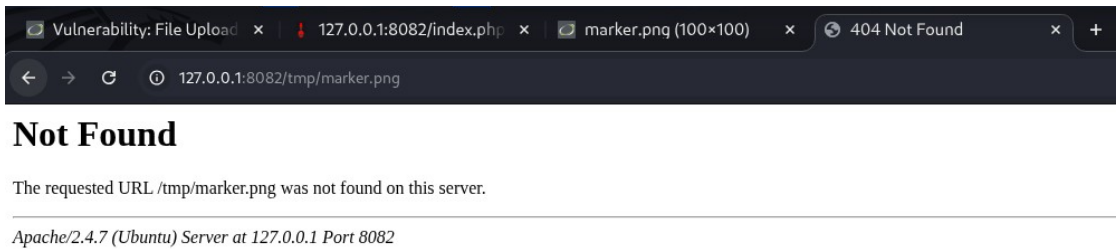
Response:

```

8 Content-Length: 51887
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html
12
13 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
14 "http://www.w3.org/TR/1999/REC-html401-19991224/loose.dtd">
15 <html>
16 <head>
17 <link rel="shortcut icon" href="/images/favicon.ico" type="image/x-icon" />
18 <link rel="stylesheet" type="text/css" href="/styles/global-styles.css" />
19 <link rel="stylesheet" type="text/css" href="/styles/ddsmoothmenu/ddsmoothmenu.css" />
20 <link rel="stylesheet" type="text/css" href="/styles/ddsmoothmenu/ddsmoothmenu-v.css" />
21
22 <script type="text/javascript" src="/javascript/bookmark-site.js">
23 </script>
24 <script type="text/javascript" src="/javascript/ddsmoothmenu/ddsmoothmenu.js">
25 </script>
26 <script type="text/javascript" src="/javascript/ddsmoothmenu/jquery.min.js">
27 </script>
28
29 * Smooth Navigational Menu- (c) Dynamic Drive DHTML code library
30 (www.dynamicdrive.com)
31 * This notice MUST stay intact for legal use
32 * Visit Dynamic Drive at http://www.dynamicdrive.com/ for full source code
33
34 <script type="text/javascript">
35 ddsmoothmenu.init({
36   mainmenuid: "smoothmenu1", //menu DIV id
37   orientation: 'v', //Horizontal or vertical menu: Set to "h" or "v"
38   classname: 'ddsmoothmenu', //class added to menu's outer DIV
39   //customtheme: ["#cccc44", "#cccccc"],
40   contentsource: "markup" // "markup" or ["container_id", "path_to_menu_file"]
41 });
42 </script>
43
44 <script type="text/javascript">
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Evidence E8 — Burp capture of the Mutillidae upload POST, showing a client-supplied UPLOAD_DIRECTORY field (/tmp).



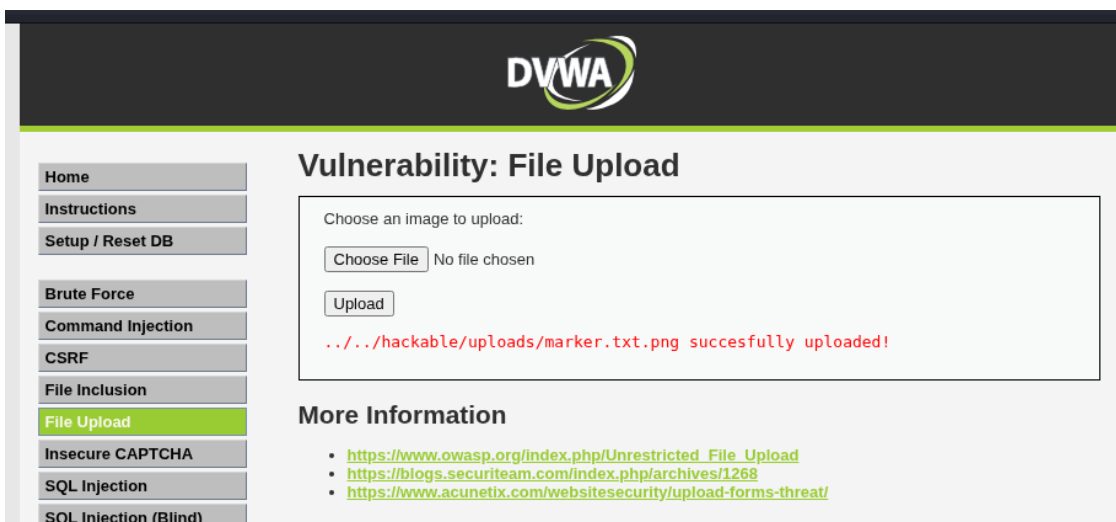
Evidence E9 — Attempted retrieval of the Mutillidae-stored file at /tmp/marker.png returns 404 Not Found, since /tmp lies outside the webroot.

5.3 Phase 2 Conclusion

Observation	DVWA	Mutillidae
Upload accepted	Yes	Yes
Stored path	hackable/uploads/marker.png	/tmp/marker.png
Directly retrievable via browser	Yes (200 OK, renders)	No (404 Not Found)
Filename preserved	Yes	Yes

6. Phase 3 — Validation Comparison (Mismatch Test)

marker.txt was duplicated and renamed to marker.txt.png — a harmless file whose actual content (plain text) does not match its declared extension (.png). This file was uploaded to both applications to determine where, if anywhere, validation is enforced.



Evidence E10 — DVWA accepts the mismatched file without any rejection or warning.

 **OWASP Mutillidae II: Keep Calm and Pwn On**

Version: 2.6.62 Security Level: 0 (Hosed) Hints: Enabled (1 - Try easier) Logged In User: **admin** (admin)

[Home](#) | [Logout](#) | [Toggle Hints](#) | [Show Popup Hints](#) | [Toggle Security](#) | [Enforce SSL](#) | [Reset DB](#) | [View Log](#) | [View Captured Data](#)

Upload a File

File uploaded to /tmp/phpWpOSn8
File moved to /tmp/marker.txt.png
Validation not performed

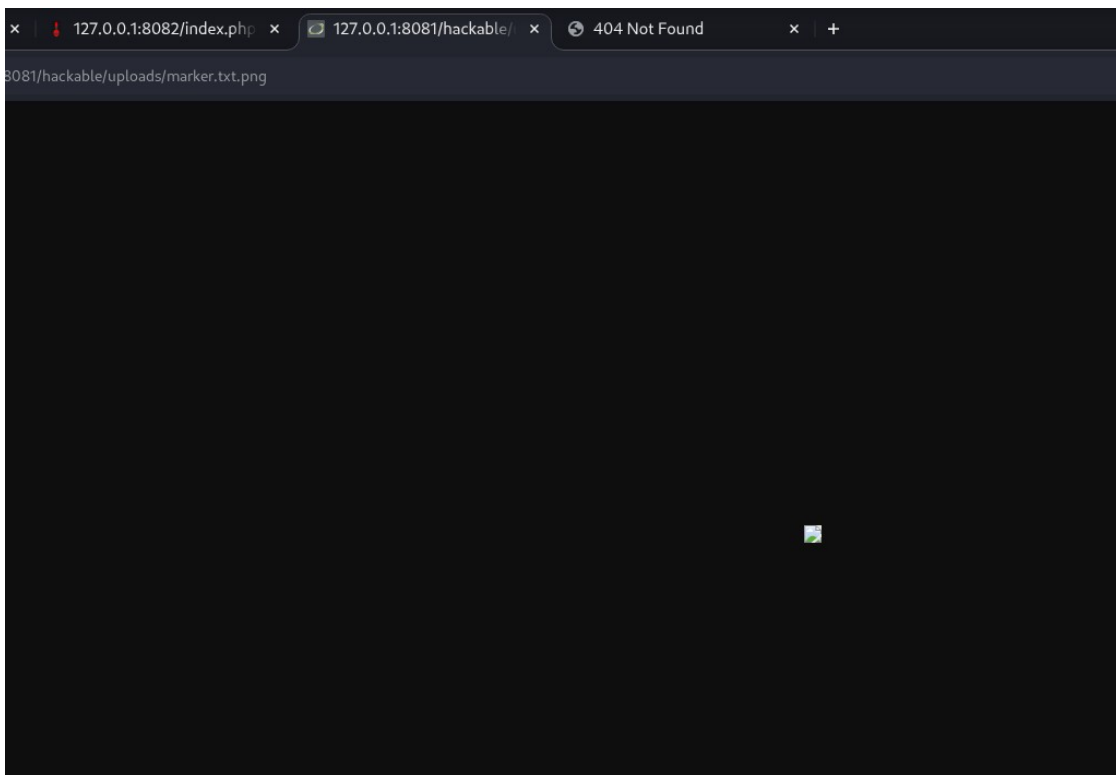
Original File Name	marker.txt.png
Temporary File Name	/tmp/phpWpOSn8
Permanent File Name	/tmp/marker.txt.png
File Type	image/png
File Size	60 Bytes

Please choose file to upload

Filename



Evidence E11 — Mutillidae accepts the file and reports File Type: image/png, despite the underlying content being plain text; the page explicitly states "Validation not performed."



Evidence E12 — Retrieving the DVWA-stored mismatch file returns HTTP 200, but the browser cannot render it as an image (broken-image icon), proving the stored bytes are not a valid PNG.

6.1 Phase 3 Conclusion

Both applications accepted a file whose actual content did not match its extension or declared MIME type, and neither performed content-based (magic-byte) validation. In Mutillidae, the reported "File Type: image/png" value was taken directly from the client-supplied Content-Type header rather than inspected server-side. In DVWA, the file was stored and served with a 200 OK response but rendered as a broken image, confirming the server accepted it without verifying actual content. Validation, where it exists at all, is extension/label-based and was not meaningfully enforced on either the client or the server at default security settings.

This directly addresses Analysis Question 1 ("Why is browser-only upload validation insufficient?"): because the server itself trusted a client-controlled label instead of inspecting file content, any client-side (browser) check is trivially bypassed by crafting the HTTP request directly, as demonstrated using Burp Suite throughout this assessment.

7. Phase 4 — Storage Assessment

Consolidating evidence from Phases 2 and 3, the following storage-handling characteristics were determined for each application.

Question	DVWA	Mutillidae
Renamed on upload?	No — original filename retained	No — original filename retained
Access-controlled?	No — publicly retrievable, no authentication required for the file itself	Not directly retrievable, but only because /tmp is not served by the web server
Stored outside executable/web-accessible path?	No — stored inside hackable/uploads/, within the webroot	Yes — stored in /tmp, outside the webroot
Storage location determined by	Server (fixed path)	Client — UPLOAD_DIRECTORY is an attacker-controlled POST field

7.1 Phase 4 Conclusion

DVWA's storage handling is weaker on the retrieval dimension: because uploads are placed inside the public webroot with no renaming or access control, any file accepted by the upload validation logic (Phase 3 showed this is effectively none at low/medium security) is immediately and publicly retrievable. Mutillidae is comparatively safer on this specific test because /tmp happens to fall outside the web-served directory, but this safety is incidental rather than by design: the application allows the client to dictate the destination directory via the UPLOAD_DIRECTORY field. If that field were pointed at a web-accessible path instead of /tmp, the same retrieval risk demonstrated on DVWA would likely apply to Mutillidae as well. This scenario was not tested directly, in line with the requirement to avoid unrelated-file access and to keep testing minimal-impact.

8. Phase 5 — Command Baseline

Normal, expected input was submitted to each application's diagnostic feature to establish a baseline of legitimate behaviour before any injection was attempted.

8.1 DVWA Command Injection — Baseline

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=6.294 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=1.927 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.116 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.217 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.116/2.139/6.294/2.505 ms
```

More Information

- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_Injection

Evidence E13 — DVWA ping baseline using input 127.0.0.1, showing normal ICMP statistics.

Request

```
1 POST /vulnerabilities/exec/ HTTP/1.1
2 Host: 127.0.0.1:8081
3 Content-Length: 26
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="139", "Not;A=Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1:8081
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1:8081/vulnerabilities/exec/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=9pm048dn1q19ullo6bnsa7moe3; security=low; showhints=1; username=admin; uid=24
21 Connection: keep-alive
22 ip=127.0.0.1&Submit=Submit
```

Response

```
1 HTTP/1.1 200 OK
2 Date: Thu, 27 Aug 2026 10:44:55 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Tue, 23 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 4599
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
14 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
15
16 <html xmlns="http://www.w3.org/1999/xhtml">
17
18 <head>
19 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
20
21 <title>
22 Vulnerability: Command Injection :: Damn Vulnerable Web Application (DVWA)
23 v1.0 *Development*
24 </title>
25
26 <link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
27 <link rel="icon" type="image/ico" href="../../favicon.ico" />
28 <script type="text/javascript" src="../../dvwa/js/dvwaPage.js">
29 </script>
30 </head>
31
32 <body class="home">
33 <div id="container">
```

Evidence E14 — Burp capture of the DVWA baseline request: POST /vulnerabilities/exec/ with parameter ip=127.0.0.1.

8.2 Mutillidae DNS Lookup — Baseline

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cool
92	http://127.0.0.1:8082	POST	/index.php?page=dns-lookup.php			302	439	text/html	php				127.0.0.1	user

Request	Response
<pre> 1 POST /index.php?page=dns-lookup.php HTTP/1.1 2 Host: 127.0.0.1:8082 3 Content-Length: 61 4 Cache-Control: max-age=0 5 sec-ch-ua: "Chromium";v="139", "Not;A=Brand";v="99" 6 sec-ch-ua-mobile: ?0 7 sec-ch-ua-platform: "Linux" 8 Accept-Language: en-US,en;q=0.9 9 Origin: http://127.0.0.1:8082 10 Content-Type: application/x-www-form-urlencoded 11 Upgrade-Insecure-Requests: 1 12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 14 Sec-Fetch-Site: same-origin 15 Sec-Fetch-Mode: navigate 16 Sec-Fetch-User: ?1 17 Sec-Fetch-Dest: document 18 Referer: http://127.0.0.1:8082/index.php?page=dns-lookup.php 19 Accept-Encoding: gzip, deflate, br 20 Cookie: PHPSESSID=9pm048dnlq19u1o6bnsa7moe3; security=low; showhints=1; username=admin; uid=24 21 Connection: keep-alive 22 23 target_host=127.0.0.1&dns-lookup-php-submit-button=Lookup+DNS </pre>	<pre> 46 </script> 47 </head> 48 <body onload="onLoadOfBody(this);"> 49 <table class="main-table-frame" border="1px" cellspacing="0px" cellpadding= 50 "0px"> 51 <tr> 52 <td bgcolor="#ccccff" align="center" colspan="7"> 53 <table width="100%"> 54 <tr> 55 <td style="text-align:center;"> 56 58 61 OWASP Mutillidae II: Keep Calm and Pwn On 62 63 </td> 64 </tr> 65 </table> 66 </td> 67 </tr> 68 <tr> 69 <td bgcolor="#ccccff" align="center" colspan="7"> 70 71 Version: 2.6.62 72 73 75 Security Level: 0 (Hosed) 76 77 79 Hints: Enabled (1 - Try easier) 80 81 83 Logged In User: 84 admin 85 86 (admin) 87 88 </td> 89 </tr> </pre>

Evidence E15 — Mutillidae DNS Lookup baseline result page.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cool
92	http://127.0.0.1:8082	POST	/index.php?page=dns-lookup.php			302	439	text/html	php				127.0.0.1	user

Request	Response
<pre> 1 POST /index.php?page=dns-lookup.php HTTP/1.1 2 Host: 127.0.0.1:8082 3 Content-Length: 61 4 Cache-Control: max-age=0 5 sec-ch-ua: "Chromium";v="139", "Not;A=Brand";v="99" 6 sec-ch-ua-mobile: ?0 7 sec-ch-ua-platform: "Linux" 8 Accept-Language: en-US,en;q=0.9 9 Origin: http://127.0.0.1:8082 10 Content-Type: application/x-www-form-urlencoded 11 Upgrade-Insecure-Requests: 1 12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 14 Sec-Fetch-Site: same-origin 15 Sec-Fetch-Mode: navigate 16 Sec-Fetch-User: ?1 17 Sec-Fetch-Dest: document 18 Referer: http://127.0.0.1:8082/index.php?page=dns-lookup.php 19 Accept-Encoding: gzip, deflate, br 20 Cookie: PHPSESSID=9pm048dnlq19u1o6bnsa7moe3; security=low; showhints=1; username=admin; uid=24 21 Connection: keep-alive 22 23 target_host=127.0.0.1&dns-lookup-php-submit-button=Lookup+DNS </pre>	<pre> 1 HTTP/1.1 200 OK 2 Date: Thu, 27 Aug 2026 10:48:33 GMT 3 Server: Apache/2.4.7 (Ubuntu) 4 X-Powered-By: PHP/5.5.9-1ubuntu4.25 5 Logged-In-User: admin 6 X-XSS-Protection: 0 7 Vary: Accept-Encoding 8 Content-Length: 52286 9 Keep-Alive: timeout=5, max=100 10 Connection: Keep-Alive 11 Content-Type: text/html 12 13 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" 14 "http://www.w3.org/TR/1999/REC-html401-19991224/loose.dtd"> 15 <html> 16 <head> 17 <link rel="shortcut icon" href="/images/favicon.ico" type="image/x-icon" /> 18 <link rel="stylesheet" type="text/css" href="/styles/global.styles.css" /> 19 <link rel="stylesheet" type="text/css" href="/styles/ddsmoothmenu/ddsmoothmenu.css" /> 20 <link rel="stylesheet" type="text/css" href="/styles/ddsmoothmenu/ddsmoothmenu-v.css" /> 21 <script type="text/javascript" src="/javascript/bookmark-site.js"> 22 </script> 23 <script type="text/javascript" src="/javascript/ddsmoothmenu/ddsmoothmenu.js"> 24 </script> 25 <script type="text/javascript" src="/javascript/ddsmoothmenu/jquery.min.js"> 26 </script> 27 /***** 28 * Smooth Navigational Menu- (c) Dynamic Drive DHTML code library 29 (www.dynamicdrive.com) 30 * This notice MUST stay intact for legal use 31 * Visit Dynamic Drive at http://www.dynamicdrive.com/ for full source code 32 *****/ 33 <script type="text/javascript"> 34 ddsmoothmenu.init({ 35 mainmenuid: "smoothmenu1", //menu DIV id 36 orientation: 'v', //Horizontal or vertical menu: Set to "h" or "v" 37 classname: 'ddsmoothmenu', //class added to menu's outer DIV 38 //customtheme: ["#cccc44", "#cccccc"], 39 contentsource: "backin" //backin or !container id "path to menu file" 40 }); </pre>

Evidence E16 — Burp capture of the Mutillidae baseline request: POST /index.php?page=dns-lookup.php with parameter target_host=127.0.0.1.

8.3 Affected Parameters Identified

- DVWA Command Injection: parameter "ip"
- Mutillidae DNS Lookup: parameter "target_host"

9. Phase 6 — Minimal Execution Proof

A harmless command-separator payload (127.0.0.1; echo LAB2TEST) was submitted to the ip and target_host parameters respectively. If the string "LAB2TEST" appears in the output, this proves the backend concatenated user input directly into a shell command rather than passing it as a safe, isolated argument.

9.1 DVWA

Vulnerability: Command Injection

Ping a device

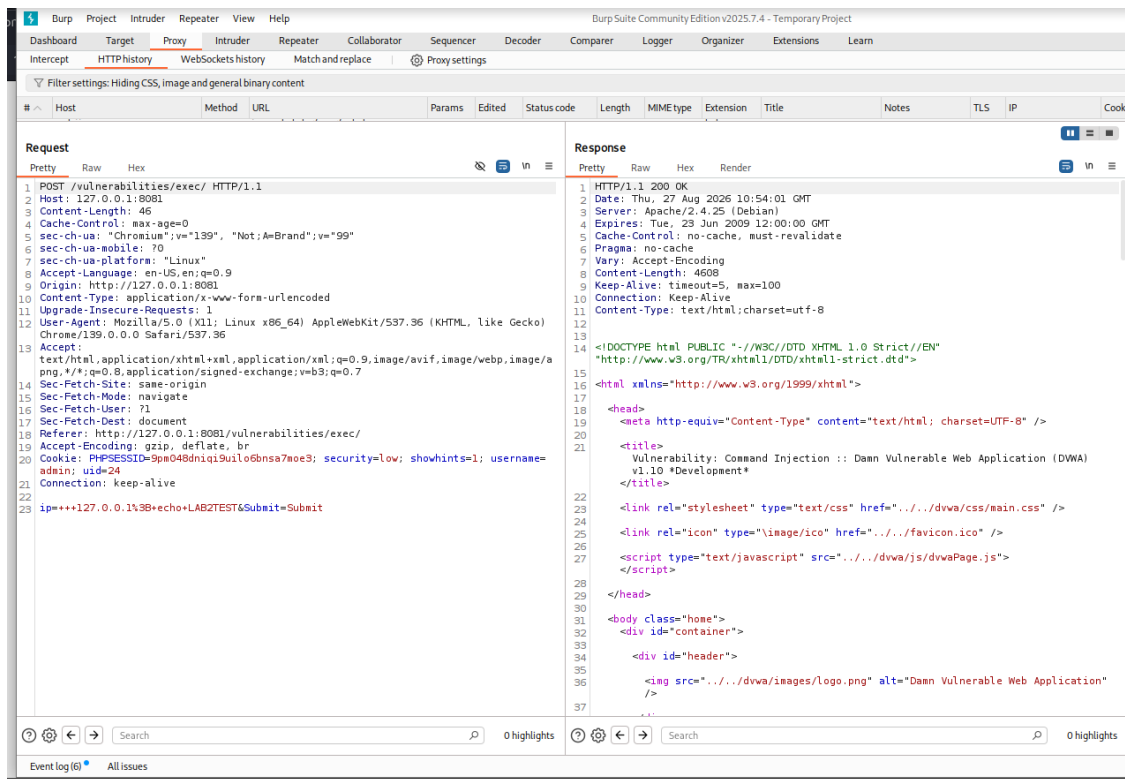
Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.242 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.081 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.189 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.112 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.081/0.156/0.242/0.063 ms
LAB2TEST
```

More Information

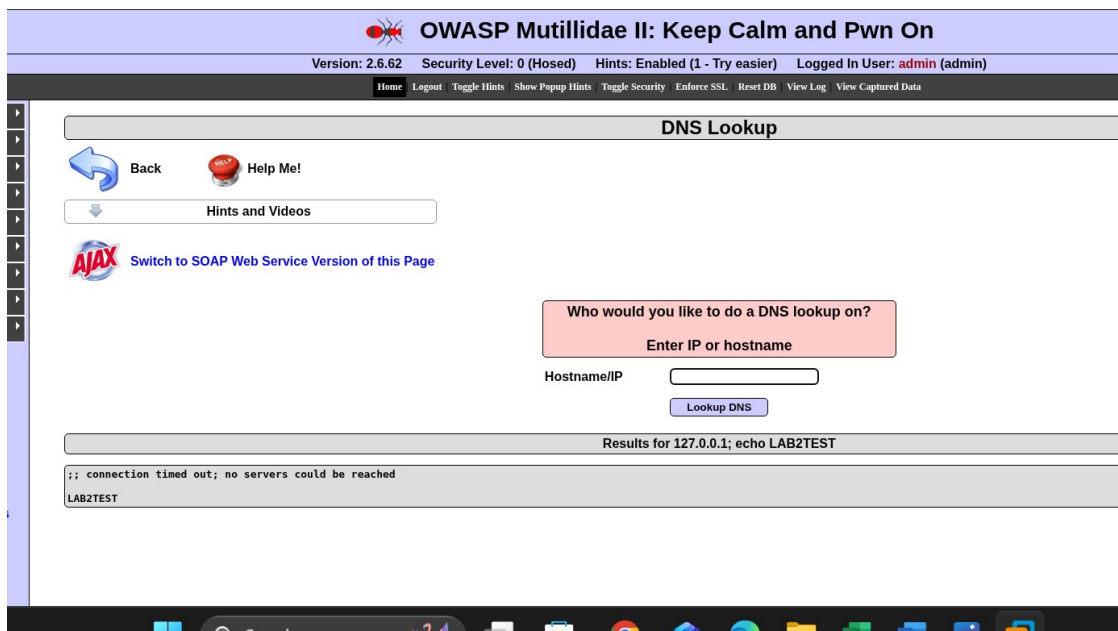
- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_Injection

Evidence E17 — DVWA output shows normal ping statistics followed by "LAB2TEST" on its own line, an unrelated string with no relationship to ping.

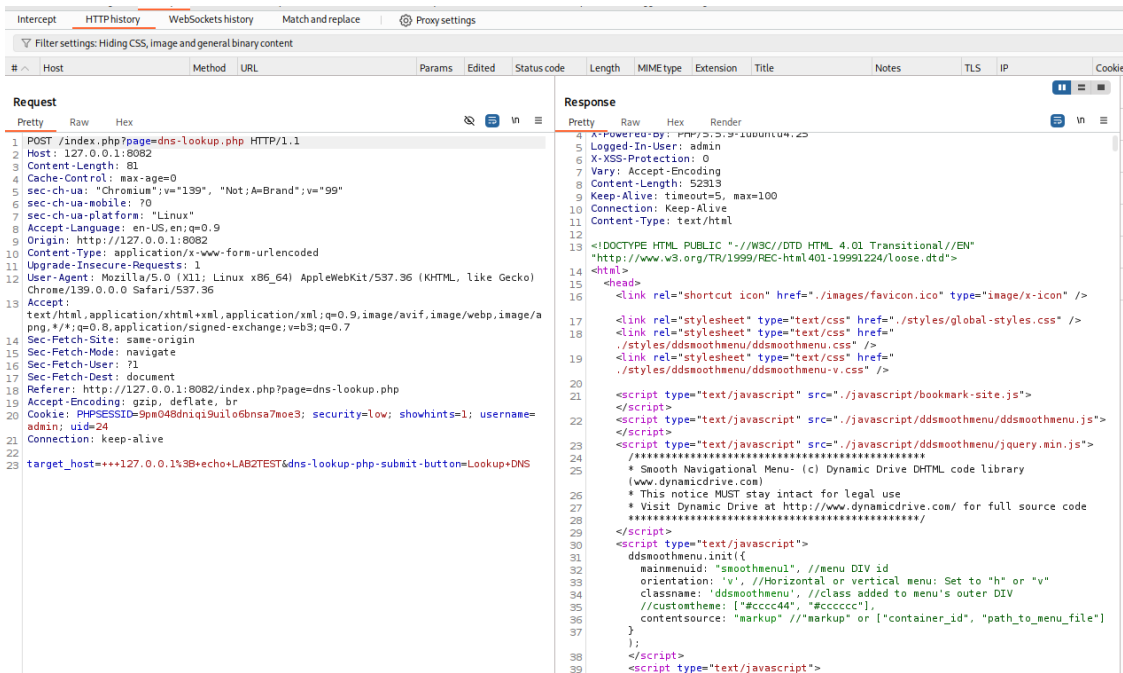


Evidence E18 — Burp capture showing the injected parameter: `ip=127.0.0.1%3B+echo+LAB2TEST`.

9.2 Mutillidae



Evidence E19 — Mutillidae result page: the DNS lookup itself fails ("connection timed out"), but "LAB2TEST" still prints, proving the echo command executed independently of the DNS tool.



Evidence E20 — Burp capture showing the injected parameter: `target_host=127.0.0.1%3B+echo+LAB2TEST`.

9.3 Phase 6 Conclusion

Injecting `"; echo LAB2TEST"` into both the `ip` parameter (DVWA) and `target_host` parameter (Mutillidae) caused the string "LAB2TEST" to appear in the output — a value with no relationship to a ping or DNS lookup result. This proves the backend concatenated user input directly into a shell command string rather than passing it as a safe, isolated argument to the diagnostic binary; the semicolon was interpreted as a shell command separator, allowing arbitrary additional command execution.

This addresses Analysis Question 2 ("What proves that command input was treated as executable structure rather than data?"): the appearance of output from an entirely unrelated, injected command (`echo`) — output that could not have been produced by `ping` or `nslookup` under any normal input — is direct proof that the shell interpreted a structural control character (`;`) rather than treating the whole input string as inert data.

10. Phase 7 — Impact Assessment

The execution context of the injected commands was identified using the harmless, read-only `whoami` command, without accessing any files, credentials, or systems beyond this single identity check.



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection**
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript
- DVWA Security
- PHP Info

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.084 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.206 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.079 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.145 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.079/0.129/0.206/0.052 ms
www-data
```

More Information

- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_Injection

Evidence E21 — DVWA: injecting "; whoami" appends www-data to the ping output, identifying the web server's process owner.


OWASP Mutillidae II: Keep Calm and Pwn On

Version: 2.6.62 Security Level: 0 (Hosed) Hints: Enabled (1 - Try easier) Logged In User: admin (admin)

[Home](#) [Logout](#) [Toggle Hints](#) [Show Popup Hints](#) [Toggle Security](#) [Enforce SSL](#) [Reset DB](#) [View Log](#) [View Captured Data](#)

DNS Lookup

 Back
 Help Me!

 [Hints and Videos](#)

 [Switch to SOAP Web Service Version of this Page](#)

Who would you like to do a DNS lookup on?

Enter IP or hostname

Hostname/IP:

Results for 127.0.0.1; whoami

```
;; connection timed out; no servers could be reached
www-data
```

Evidence E22 — Mutillidae: the same whoami injection confirms the DNS Lookup feature also executes as www-data.

10.1 Observed vs. Potential Impact

Dimension	Observed (what was actually done)	Potential (what the same access could enable)
Execution context	Confirmed www-data on both applications	Any command runs as www-data — a low-privilege but still functionally powerful account on a typical LAMP host

Dimension	Observed (what was actually done)	Potential (what the same access could enable)
Actions taken	Printed a harmless marker string; printed the current username	Read/exfiltrate any file readable by www-data (application source, configuration files, embedded database credentials)
Persistence	None created	A web shell, cron entry, or SSH key could be planted if any web-writable directory exists
Privilege escalation	Not attempted	Possible if SUID binaries, sudo misconfigurations, or kernel vulnerabilities exist on the host — not tested, out of scope
Lateral movement	None performed	If www-data can reach internal services or holds credentials, injected commands could pivot to other systems

10.2 Phase 7 Conclusion

Although the test only produced a harmless marker string and a username, the confirmed ability to execute arbitrary commands as www-data means the true severity of this vulnerability is "remote code execution as the web server process owner." The ceiling of potential harm is bounded only by what www-data is permitted to do on the underlying host, not by the limited scope of the commands actually run during this assessment. This is the practical basis for Analysis Question 3 ("Why is minimal proof sufficient for both weaknesses?"): a single, safe, distinguishing side-effect — an unrelated string in the output, or successful retrieval of a content-mismatched upload — is sufficient to conclusively demonstrate that a vulnerability class exists, without needing to realise its full potential impact.

11. Phase 8 — Secure Comparison and Retest

The key upload-mismatch and command-injection tests were repeated at DVWA's Medium and High security levels to evaluate what changes as security posture is strengthened.

11.1 DVWA — Medium Security

Vulnerability: File Upload

Choose an image to upload:

No file chosen

.../hackable/uploads/marker.txt.png succesfully uploaded!

More Information

- https://www.owasp.org/index.php/Unrestricted_File_Upload
- <https://blogs.securiteam.com/index.php/archives/1268>
- <https://www.acunetix.com/websecurity/upload-forms-threat/>

Username: admin
Security Level: medium
PHPIDS: disabled

Evidence E23 — At Medium security, marker.txt.png is still accepted ("succesfully uploaded"), identical to the Low-security result.

Request

```

1 GET /hackable/uploads/marker.txt.png HTTP/1.1
2 Host: 127.0.0.1:8081
3 sec-ch-ua: "Chromium";v="139", "Not;A=Brand";v="99"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Cookie: PHPSESSID=9pm048dniq19u1o6bnsa7moe3; showhints=1; username=admin; uid=24; security=medium
16 If-None-Match: "3c-65a04ee4b5910"
17 If-Modified-Since: Thu, 27 Aug 2026 10:38:04 GMT
18 Connection: keep-alive
19
20

```

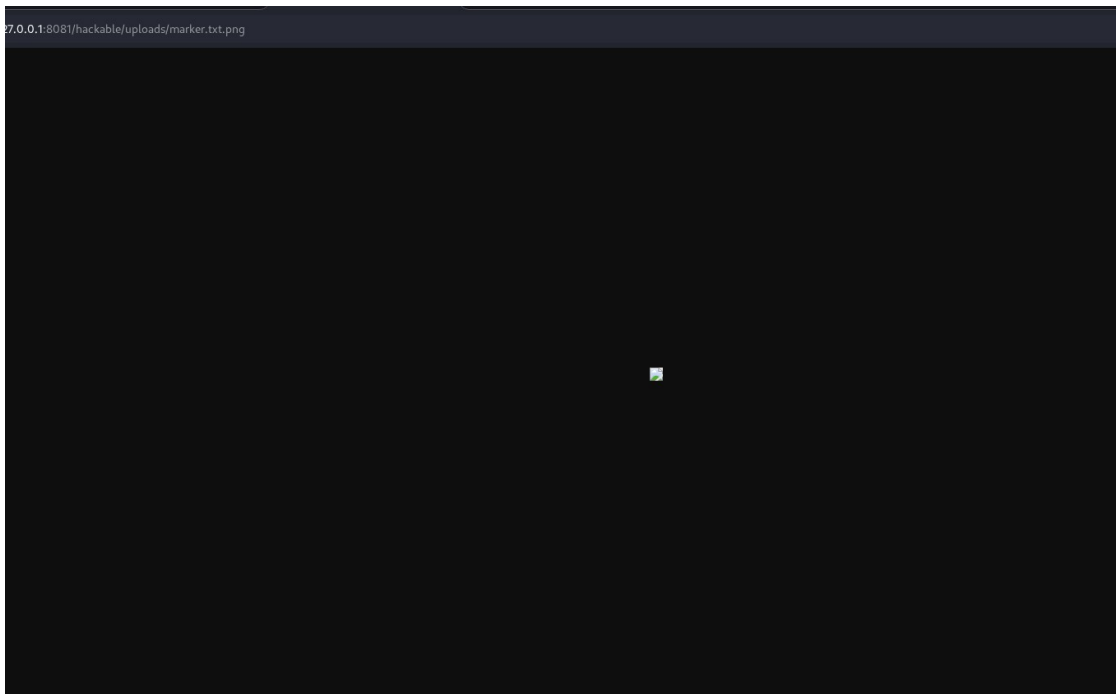
Response

```

1 HTTP/1.1 200 OK
2 Date: Fri, 28 Aug 2026 06:34:09 GMT
3 Server: Apache/2.4.25 (Debian)
4 Last-Modified: Fri, 28 Aug 2026 06:32:04 GMT
5 ETag: "3c-65a159c57c186"
6 Accept-Ranges: bytes
7 Content-Length: 60
8 Keep-Alive: timeout=5, max=100
9 Connection: Keep-Alive
10 Content-Type: image/png
11
12 Lab2 marker file - Sadeeq - Thu Aug 27 05:15:29 AM EDT 2026
13

```

Evidence E24 — Burp capture confirming security=medium cookie was active for the retest, ruling out session/level ambiguity.



Evidence E25 — The uploaded mismatch file is still directly retrievable and still fails to render, confirming no functional change from Low security.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```

PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=8.866 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.092 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.191 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.477 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.092/2.407/8.866/3.732 ms

```

More Information

- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_Injection

Username: admin

Security Level: medium

PHP Info disabled

Evidence E26 — At Medium security, submitting ";" whoami" now returns only the normal ping output — the injected command did not execute or its output was suppressed.

Request

```

1 POST /vulnerabilities/exec/ HTTP/1.1
2 Host: 127.0.0.1:8081
3 Content-Length: 36
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="119", "Not;A=Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1:8081
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1:8081/vulnerabilities/exec/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=9pw046m1q19u1o6bnsa7moE3; showhints=1; username=admin; uid=24; security=medium
21 Connection: keep-alive
22 ip=127.0.0.1%3B+whoami&Submit=Submit
23

```

Response

```

1 HTTP/1.1 200 OK
2 Date: Fri, 28 Aug 2026 06:37:04 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Tue, 28 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 4608
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
14 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
15
16 <html xmlns="http://www.w3.org/1999/xhtml">
17
18 <head>
19 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
20
21 <title>
22 Vulnerability: Command Injection :: Damn Vulnerable Web Application (DVWA)
23 v1.0 *Development*
24 </title>
25
26 <link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
27 <link rel="icon" type="image/ico" href="../../favicon.ico" />
28 <script type="text/javascript" src="../../dvwa/js/dvwaPage.js">

```

Evidence E27 — Burp capture confirming security=medium and the exact injected parameter (ip=127.0.0.1%3B+whoami) that was blocked.

11.2 DVWA — High Security

Vulnerability: File Upload

Choose an image to upload:

No file chosen

Your image was not uploaded. We can only accept JPEG or PNG images.

More Information

- https://www.owasp.org/index.php/Unrestricted_File_Upload
- <https://blogs.securiteam.com/index.php/archives/1268>
- <https://www.acunetix.com/websitesecurity/upload-forms-threat/>

File Upload

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript

DVWA Security
PHP Info
About

Logout

Username: admin
Security Level: high
PHPIDS: disabled

Evidence E28 — At High security, DVWA rejects marker.txt.png outright: "Your image was not uploaded. We can only accept JPEG or PNG images." This is the first point at which upload validation showed any improvement.

Request		Response	
Pretty	Raw	Pretty	Raw
1 POST /vulnerabilities/upload/ HTTP/1.1		1 HTTP/1.1 200 OK	
2 Host: 127.0.0.1:8081		2 Date: Fri, 28 Aug 2026 06:40:44 GMT	
3 Content-Length: 456		3 Server: Apache/2.4.25 (Debian)	
4 Cache-Control: max-age=0		4 Expires: Tue, 23 Jun 2009 12:00:00 GMT	
5 sec-ch-ua: "Chromium";v="139", "Not;A=Brand";v="99"		5 Cache-Control: no-cache, must-revalidate	
6 sec-ch-ua-mobile: 70		6 Pragma: no-cache	
7 sec-ch-ua-platform: "Linux"		7 Vary: Accept-Encoding	
8 Accept-Language: en-US,en;q=0.9		8 Content-Length: 4275	
9 Origin: http://127.0.0.1:8081		9 Keep-Alive: timeout=5, max=100	
10 Content-Type: multipart/form-data; boundary=---WebKitFormBoundary8FhydWnzQXORn6CC		10 Connection: Keep-Alive	
11 Upgrade-Insecure-Requests: 1		11 Content-Type: text/html; charset=utf-8	
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36		12	
13 Accept:		13	
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		14 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN	
14 Sec-Fetch-Site: same-origin		"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">	
15 Sec-Fetch-Mode: navigate		15	
16 Sec-Fetch-User: ?1		16 <html xmlns="http://www.w3.org/1999/xhtml">	
17 Sec-Fetch-Dest: document		17	
18 Referer: http://127.0.0.1:8081/vulnerabilities/upload/		18 <head>	
19 Accept-Encoding: gzip, deflate, br		19 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />	
20 Cookie: PHPSESSID=9pm048dn1q19u1o6bnsa7moe3; showhints=1; username=admin; uid=24; security=high		20 <title>	
21 Connection: keep-alive		Vulnerability: File Upload :: Damn Vulnerable Web Application (DVWA) v1.10	
22		*Development*	
23		</title>	
24		24 <link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />	
25		25 <link rel="icon" type="image/ico" href="../../favicon.ico" />	
26		26 <script type="text/javascript" src="../../dvwa/js/dvwaPage.js">	
27		27 </script>	
28		28 </head>	
29		29 <body class="home">	
30		30 <div id="container">	
31		31	
32		32 <div id="header">	
33		33	
34		34	
36		36	
37			
38			

Evidence E29 — Burp capture confirming security=high was active during the successful rejection.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: Command Injection

Ping a device

Enter an IP address:

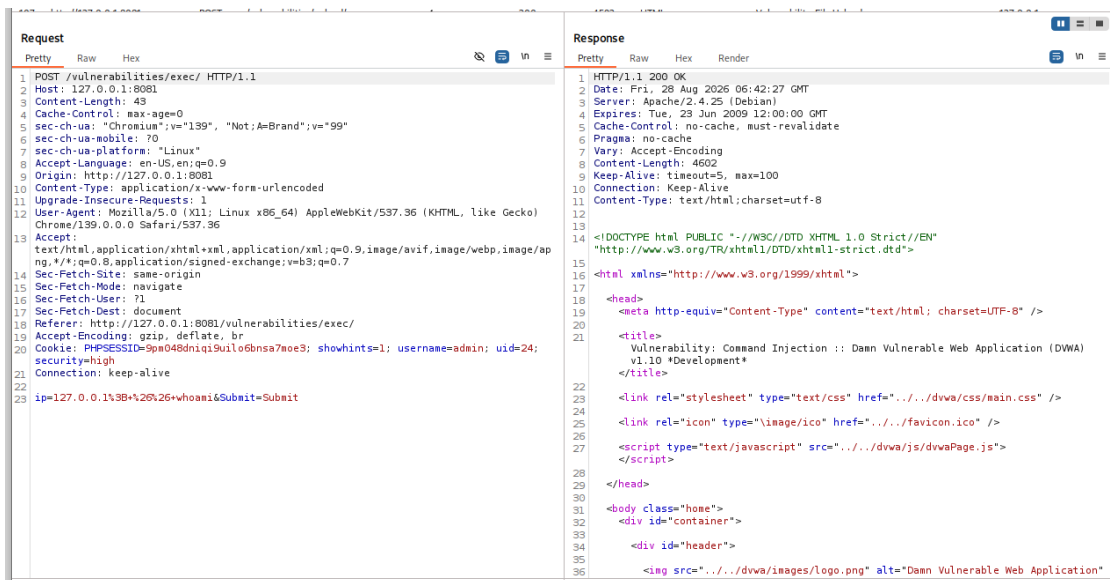
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.288 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.200 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.208 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.134 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.134/0.207/0.288/0.055 ms

More Information

- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_Injection

Username: admin
Security Level: high
PHPIDS: disabled

Evidence E30 — At High security, an obfuscated injection attempt still returns only clean ping output; command-injection filtering remains effective.



Evidence E31 — Burp capture confirming security=high and the exact payload attempted.

11.3 Consolidated Comparison

Test	Low	Medium	High
Upload mismatch (marker.txt.png)	Accepted, stored, retrievable	Accepted, stored, retrievable — no change	Rejected — "We can only accept JPEG or PNG images"
Command injection (; whoami)	Executed — www-data printed	Blocked — only ping output returned	Blocked — only ping output returned

11.4 Phase 8 Conclusion

Command-injection filtering proved effective starting at Medium security and remained effective at High, blocking the semicolon command separator even when combined with additional obfuscation. File upload validation, in contrast, showed no improvement at Medium and only became effective at High, where DVWA began enforcing genuine content-type checking for the first time. This reveals that DVWA's single "security level" setting does not harden every module uniformly or at the same rate: command-injection controls activate one level earlier than upload controls.

This answers Analysis Question 4 ("Which control produced the greatest improvement in stronger mode?"): the single greatest improvement observed was the File Upload module's transition from Medium to High, where a previously completely permissive control (accepting any file regardless of content) became an enforced allowlist check for the first time. The broader lesson is that a global security-level label does not guarantee equal protection across every feature of an application — each attack surface must be independently verified rather than assumed secure because "security" is set to a higher value elsewhere.

12. Phase 9 — Remediation Recommendations

12.1 File Upload Remediation

Weakness Observed	Root Cause	Recommended Remediation
Filename/content mismatch accepted	Server trusted the client-supplied extension/MIME type	Validate file content via magic-byte/signature inspection server-

Weakness Observed	Root Cause	Recommended Remediation
	instead of inspecting actual file content	side (e.g. confirm the file begins with the PNG signature) independently of filename or client-supplied Content-Type
No effective content-type verification at Low/Medium	Missing server-side allowlist enforcement	Enforce a strict allowlist of permitted file types based on verified content, not extension; reject anything that does not match
Original filename preserved on storage	Predictable, attacker-influenced filenames stored as-is	Rename uploaded files server-side to a randomly generated identifier (e.g. UUID); never use the client-supplied filename for storage
Upload stored inside public webroot (DVWA)	Upload directory is directly web-accessible	Store uploads outside the webroot, or in a directory with script execution explicitly disabled; serve files only through a controlled download handler
Client-controlled upload destination (Mutillidae)	Server trusts a client-supplied UPLOAD_DIRECTORY parameter	Hardcode the destination path server-side; never allow client input to influence storage location
Single point of validation	Client-side checks are the only barrier in practice, easily bypassed via direct request crafting	Validate at every layer: client-side for user experience only, mandatory server-side validation, and periodic post-storage re-scanning

12.2 Command Execution Remediation

Weakness Observed	Root Cause	Recommended Remediation
User input concatenated into a shell command	Application likely uses system()/exec()/shell-invoking calls with unsanitised input	Avoid shelling out entirely; use language-native libraries for the required function (e.g. a DNS resolution library instead of nslookup; an ICMP library instead of ping)
Blacklist-based filtering only (Medium/High)	Blacklisting individual characters (;, &&) is inherently incomplete; other separators/encodings may bypass it	Replace blacklisting with strict allowlist input validation (e.g. a valid IPv4/hostname regular expression); reject anything not matching
Command executes as www-data	No privilege separation between the web process and any subprocess it spawns	Run the web application under a least-privilege account; if shelling out is unavoidable, execute within a sandboxed/restricted environment
No structural separation between command and	Command built as a single string rather than as discrete arguments	Use parameterised/argument-array execution APIs (e.g. execve-style)

Weakness Observed	Root Cause	Recommended Remediation
argument		calls with an argument array) so shell metacharacters cannot be reinterpreted

12.3 Cross-Cutting Root Cause

Both vulnerability classes share a common root cause: trusting client-supplied data to determine server-side behaviour — whether that is a file's declared type, name, or destination, or a command's structure. The most durable fix in both cases is architectural rather than cosmetic: validate content (not labels) for uploads, and eliminate shell invocation entirely (rather than merely filtering its input) for command-backed features.

13. Phase 10 — Cleanup and Restoration

- All learner-created marker files (marker.txt, marker.png, marker.txt.png and their uploaded copies) were removed from both target applications.
- DVWA database/application state was reset via Setup / Reset DB.
- Mutillidae application state was reset via Reset DB.
- No listener, temporary account, scheduled task, or persistent artefact was created at any point during testing; no such items required removal.
- All tests were confirmed to have remained local to the isolated VM (127.0.0.1) and non-destructive throughout.

Cleanup was completed and confirmed by the assessor prior to compiling this report.

14. Analysis Questions

14.1 Why is browser-only upload validation insufficient?

Because in both applications tested, even the server itself did not verify actual file content — it relied on a client-supplied extension or Content-Type header. Any check performed only in the browser (e.g. JavaScript restricting the file picker) is trivially bypassed by an attacker who crafts the HTTP request directly, as was done throughout this assessment using Burp Suite. Effective validation must occur server-side and must inspect actual file content (e.g. magic bytes), not client-declared labels.

14.2 What proves that command input was treated as executable structure rather than data?

The appearance of output from an entirely unrelated, injected command. When `"; echo LAB2TEST"` was submitted, the string `"LAB2TEST"` appeared in the response even though it has no relationship to a ping or DNS lookup result. This can only occur if the semicolon was interpreted as a shell command separator rather than as a literal character within a single data value — i.e. the input altered the structure of the command being executed, not merely its data content.

14.3 Why is minimal proof sufficient for both weaknesses?

A single, safe, clearly distinguishing side-effect is enough to conclusively establish that a vulnerability class exists, without needing to realise its full potential impact. For uploads, successfully retrieving a content-mismatched file proves the absence of content validation. For command injection, an unrelated string appearing in output

proves arbitrary command execution. Escalating further (e.g. reading sensitive files, achieving persistence) would add risk without adding proof value, and is explicitly prohibited by the lab's authorised scope.

14.4 Which control produced the greatest improvement in stronger mode?

The File Upload module's transition from Medium to High security produced the greatest single improvement: a previously completely permissive control (any file accepted regardless of content) became a functioning allowlist check for the first time at High. Command-injection filtering also improved, but did so one level earlier (at Medium) and remained flat through High, meaning the marginal improvement per level was smaller once effective. This indicates that raising a global security-level setting does not harden every feature at the same rate.

15. Evidence Register

The table below records every piece of evidence gathered during this assessment, in chronological order.

Ev. #	Source	Date/Time (WAT approx.)	Action	Output / Purpose
E1	Local VM terminal	27 Aug 2026	Created marker.txt and computed SHA-256/size/type	Baseline fingerprint of harmless text marker
E2	Local VM terminal	27 Aug 2026	Generated marker.png via Python/Pillow and fingerprinted	Baseline fingerprint of harmless image marker
E3	DVWA	27 Aug 2026	Uploaded marker.png (Low security)	Confirms successful baseline upload and stored path
E4-E5	DVWA / Burp	27 Aug 2026	Captured raw request/response for baseline upload	Raw evidence of normal upload transaction
E6	DVWA (browser)	27 Aug 2026	Retrieved uploaded file directly via URL	Confirms public retrievability / content delivery
E7	Mutillidae	27 Aug 2026	Uploaded marker.png (security level 0)	Confirms baseline upload; explicitly states no validation performed
E8	Mutillidae / Burp	27 Aug 2026	Captured raw upload request	Reveals client-controlled UPLOAD_DIRECTORY field
E9	Mutillidae (browser)	27 Aug 2026	Attempted retrieval at /tmp/marker.png	Confirms 404 — file stored outside webroot
E10	DVWA	27 Aug 2026	Uploaded marker.txt.png (mismatch test)	Confirms acceptance of mismatched file, no rejection

Ev. #	Source	Date/Time (WAT approx.)	Action	Output / Purpose
E11	Mutillidae	27 Aug 2026	Uploaded marker.txt.png (mismatch test)	Confirms acceptance; "Validation not performed" message
E12	DVWA (browser)	27 Aug 2026	Retrieved mismatch file	Broken-image render proves no content validation
E13–E14	DVWA / Burp	27 Aug 2026	Submitted 127.0.0.1 to Command Injection	Normal command-feature baseline
E15–E16	Mutillidae / Burp	27 Aug 2026	Submitted 127.0.0.1 to DNS Lookup	Normal command-feature baseline
E17–E18	DVWA / Burp	27 Aug 2026	Submitted 127.0.0.1; echo LAB2TEST	Minimal execution proof — unrelated string in output
E19–E20	Mutillidae / Burp	27 Aug 2026	Submitted 127.0.0.1; echo LAB2TEST	Minimal execution proof — unrelated string in output
E21–E23	DVWA / Mutillidae / Burp	27 Aug 2026	Submitted 127.0.0.1; whoami	Execution-context identification (www-data on both)
E24–E25	DVWA / Burp	28 Aug 2026	Retested mismatch upload and ; whoami at Medium security	No change on upload; command injection blocked
E26–E27	DVWA / Burp	28 Aug 2026	Retested mismatch upload and injection at High security	Upload rejected for the first time; injection remained blocked

16. Activity Log

Step	Activity	Tool Used	Notes / Corrections
1	Created and fingerprinted marker files	bash, sha256sum, file, Python/Pillow	First marker.png attempt failed (source file path did not exist); corrected by generating a fresh PNG with Pillow
2	Baseline upload to DVWA and Mutillidae	Browser, Burp Suite (proxy/intercept)	None
3	Confirmed retrieval of both baseline uploads	Browser direct navigation	Mutillidae file not retrievable via browser due to /tmp storage — expected and

Step	Activity	Tool Used	Notes / Corrections
			documented, not an error
4	Created mismatch file (marker.txt.png) and uploaded to both apps	bash (cp), browser	None
5	Confirmed mismatch file retrieval/render behaviour	Browser direct navigation	Broken-image icon on DVWA — expected result confirming lack of content validation
6	Captured command-feature baselines on both apps	Browser, Burp Suite	None
7	Performed minimal execution proof (echo LAB2TEST)	Browser form submission, Burp Suite	None
8	Performed impact assessment (whoami)	Browser form submission, Burp Suite	None
9	Retested at DVWA Medium and High security	DVWA Security settings page, browser, Burp Suite	None
10	Compiled remediation recommendations	Manual analysis	None
11	Cleanup: removed markers, reset both application databases	DVWA Setup/Reset DB, Mutillidae Reset DB	Confirmed no persistent artefacts remained

17. Overall Conclusion

This assessment confirms that the development team's claim of secure file upload and diagnostic features does not hold at default (low) security settings in either DVWA or OWASP Mutillidae II. Both applications accepted uploaded files with mismatched content and type, and both executed operating-system commands injected through diagnostic input fields, in each case running as the www-data user. Raising DVWA's security level improved command-injection resistance from Medium onward and upload validation only from High onward, demonstrating that a single security-level setting does not uniformly protect every feature of an application. The remediations proposed in Section 12 address the underlying root cause in both cases — trusting client-supplied data (labels, filenames, command arguments) to determine server-side behaviour — rather than merely patching the specific payloads used during this assessment.

All testing was conducted within the authorised scope: no executable files were uploaded, no destructive commands were run, no persistence was created, and no unrelated systems, files, or credentials were accessed. Cleanup was performed and confirmed at the conclusion of testing.